

• tellg() devuelve la posición de ~~lectura~~ ^{lectura} actual

• tellp() " " " " escritura " "

• good() comprueba que todo esté bien } bool

• fail() comprueba si algo falló

• eof() " " si el stream encontró el final del archivo

• clear() reinicia todos los state flags

const char* → write pues no cambia bytes
char* → read

• close() hasta final de todo

• `write(char* data, std::streamsize count)`

↑
dirección del
primer byte a escribir

↑
cantidad de bytes
a escribir

sirve para escribir una cantidad
exacta de bytes desde la memoria
hacia el archivo.

recuerda que es `char` el tipo por lo que se necesita de:

`reinterpret_cast<T>(id_var)` No convierte var a algo nuevo, solo lo ve los
bytes de memoria como una secuencia de T.

• `read(char* destino, std::streamsize count)`

`read` obtiene `count` bytes del
archivo y los pone en `dest`.

• `data()` devuelve puntero al primer elemento almacenado

• `seekg()` mueve la posición usada para la próxima lectura
(`std::streampos pos`)

• `seekp()` lo mismo, pero escritura

(offset, reference)

`seekg tellg seekp tellp`

(`std::streampos pos`)
(`std::streamoff delta`, `std::ios::seekdir`)

Funciones de fstream

recordatorio: stream es una tubería/conexión del programa al archivo.

- `open(string filename, std::ios_base::openmode mode)`
- `is_open()` checks whether the stream is currently associated with a file
- `flush()` envía al archivo los bytes pendientes que el stream mantiene en el buffer interm.

Nota: Buffer es como un contenedor temporal que almacena datos mientras se producen, transfieren o consumen. O sea prepara datos antes de usarlos, como el input buffer de smash.

- `close()` cierra el archivo asociado al stream, flushing pending output.

Tipos de posicionamiento

std:: representa
streampos una posición específica dentro del flujo 
streamoff de offset. Representa Δ entre posiciones.
streamsize cantidad a transferir. write usando 2do param streamsize
read

Registros de tamaño variable

hay grandes problemas cuando
uno de los campos es variable,
ej: string nombre;

si fuera char nombre [20];

no hay problema, pues

R es constante.
 $H + iR$

esto es lo ideal y si no hay problema
en hacer un registro constante entonces
es recomendable hacerlo.

si fuera string (que por ahora nos enfocaremos
solo en eso) tocaría hacer lo que hace java,
tener unos bytes antes que indiquen el tamaño
del string (cosa, escribir `s.length()`).

beg, cur y end es desplazar el punto de referencia para las metas.

Digamos que origen $\equiv 40$ y eof $= 100$

$$\text{seekg}(20, \text{beg}) = 0 + 20$$

$$\text{seekg}(20, \text{cur}) = 40 + 20$$

$$\text{seekg}(20, \text{end}) = 100 - 20$$

registros
fijos

$$\text{offset}(i) = H + iR$$

$H =$ "cabecera"
 $R =$ tamaño 1 registro (bytes)
 $i =$ pos registro i -ésimo

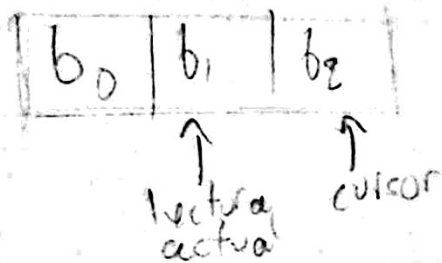
Ej:

$$H = 12$$
$$R = 32 \text{ c/u}$$
$$i \text{ en } 5$$

$$12 + 5(32) = 172$$

Posicionamiento y acceso aleatorio

Al ser un archivo una secuencia de bytes, el stream mantiene un cursor de la siguiente lectura/escritura (como el PC).



tellg(): devuelve la posición del cursor (osea la próxima lectura)
(g = get)

si devuelve 2 significa que la siguiente instrucción empieza en el offset 2

tipo: std::streampos

Lectura	Escritura	Propósito
tellg	tellp	consultar la posición
seekg	seekp	cambiar la posición

el cursor es universal, pero a excepción de `fstream`, `ifstream` y `ofstream` deben usar la rama `g` o `p` dependiendo si es `i/o`

End of File (eof)

no es bueno hacer un `while (!in.eof())`
porque solo informa si la anterior intento leer más allá del final, no predice la otra.

Entonces mejor leer primero y entrar al ciclo si la lectura fue exitosa.

```
while (in.read(reinterpret_cast<char*>(&var), sizeof(var)))
```

un int vamos

```
std::ostream& write(const char* source, std::streamsize count);
```

```
std::istream& read(char* dest, std::streamsize count);
```

↳ puntero al primer byte del objeto

tip: para objetos
trata usar tipos
de `<stdint.h>` para
tamaños consistentes

Un stream puede construirse sin abrir y luego asociarse mediante open.

`std::ifstream file; // ojo: no es un file existente` y `file` es como template, en el futuro se puede usar para otro archivo
`file.open(ruta, std::ios::binary);`

O construir y abrir de un solo

`std::ifstream entrada(ruta, std::ios::binary);` ← usualmente mejor

Modos de apertura

`std`

`ios::in`

`ios::out`

`ios::binary`

`ios::trunc`

`ios::app`

`ios::ate`

entrada

salida

evita traducciones de modo texto

elimina contenido anterior (tamaño reducido a cero)

de append. añade al final (siempre, forzosamente)

posiciona puntero al final. Puedes mover lo después, no es forzoso

Stream: abstracción mediante la cual un programa recibe o envía datos como una secuencia ordenada de unidades.

es como una tubería

archivo $\rightarrow$ bytes $\rightarrow$ programa leer

programa $\rightarrow$ bytes $\rightarrow$ archivo escribir

en binario se usa la librería `fstream` que trae consigo clases muy útiles

`std::ifstream`

archivo $\rightarrow$ programa

modo implícito
`std::ios::in`

Uso
lectura

`std::ofstream`

programa $\rightarrow$ archivo

`std::ios::out`

creación/escritura

`std::fstream`

bidireccional

`std::ios::in | std::ios::out`

lectura y actualización

`fstream` transporta unidades de tipo `char`
pues `un char == 1 byte`

no necesariamente transporta texto,
`char` solo es un contenedor.

Serialización se denomina al transformamiento de una estructura lógica a una secuencia de bytes que obedece un formato.

Deserialización: Interpretación válida de bytes para construir un nuevo objeto lógico.

Las ventajas de los archivos binarios son:

- Acceso directo: como un array, si todos los registros tienen tamaño fijo, se puede calcular exactamente con aritmética de punteros la posición de cada uno.
 - Se puede sobrescribir un registro sin reescribir el resto.
 - Usualmente el tamaño es menor.
 - Sin necesidad de parsear texto.
- A veces

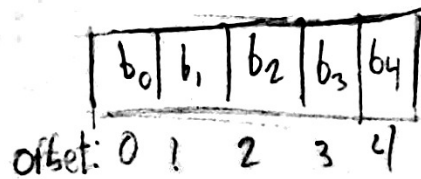
En resumen, mejor performance

Archivos Binarios

Un archivo es un objeto persistente administrado por el OS y contiene una secuencia finita de bytes.

$$F = \{b_0, b_1, b_2, \dots, b_{n-1}\}$$

La posición de un byte dentro del archivo se llama offset.
se usa como punto de referencia b_0 .



$$\text{dir_final} = \text{dir_base} + \text{offset}$$

usualmente $\text{dir_final} = \text{offset}$

Un archivo binario es un archivo cuyos bytes se interpretan según una estructura de finida por el programador.

En código es aconsejable comentar el formato de la siguiente forma

1º Formato: nombre_archivo.nombre_extensión (size archivo)

T var_1;

T var_2;

⋮

T var_n;

*/